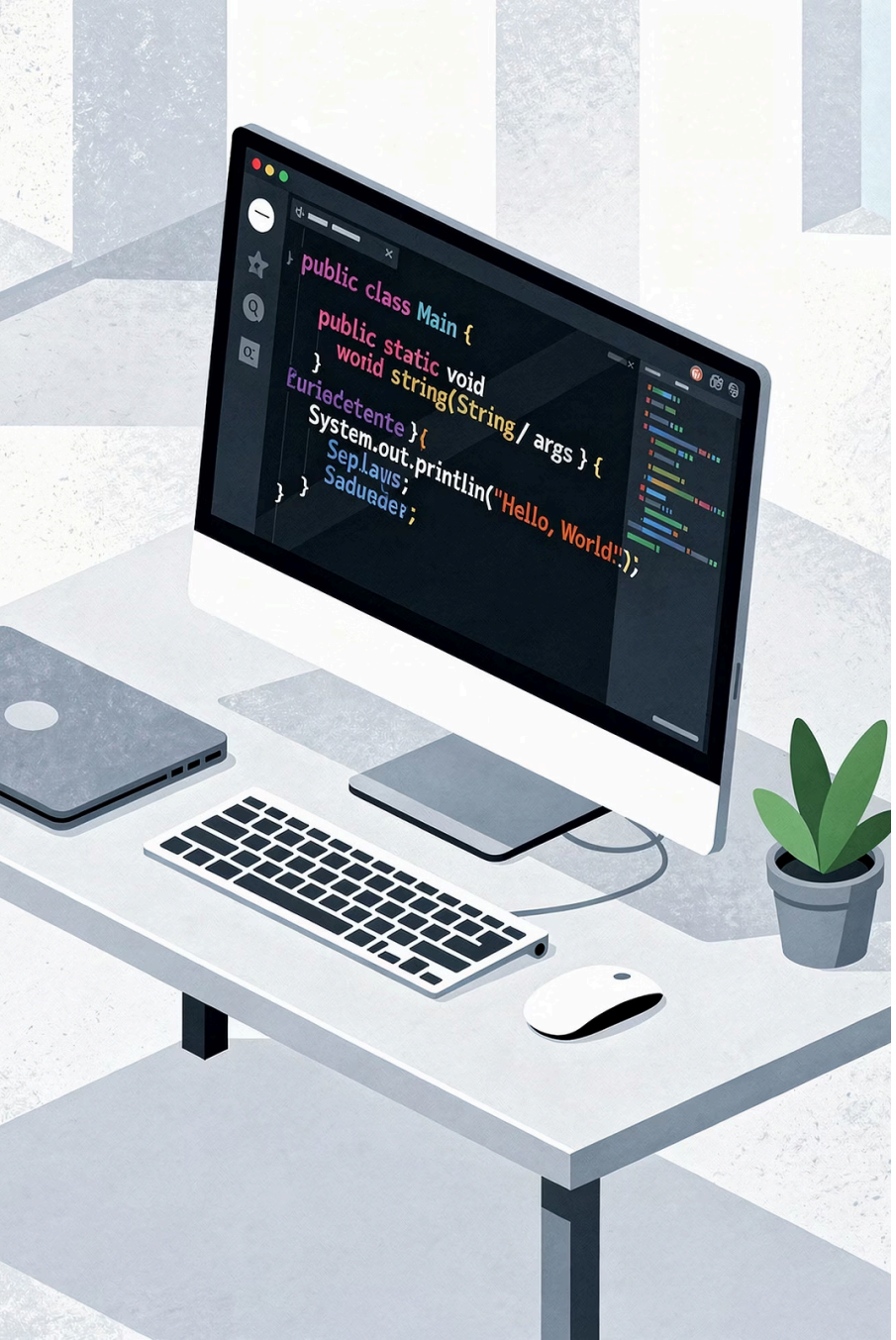


Fundamentos de Java: Estándares de Programación y Depuración

Una guía completa de buenas prácticas, estándares de cátedra y técnicas profesionales para el desarrollo en Java.

PEDAGOGÍA TÉCNICA JAVA

Contenido del Curso

01

Bloque 1: Entorno y Gestión

Configuración de Java LTS, NetBeans y gestión de proyectos.

02

Bloque 2: Comentarios y Sintaxis

Documentación, debugging con comentarios y variables.

03

Bloque 3: Estándar del Scanner

Entrada de datos, el problema del búfer y la solución definitiva.

04

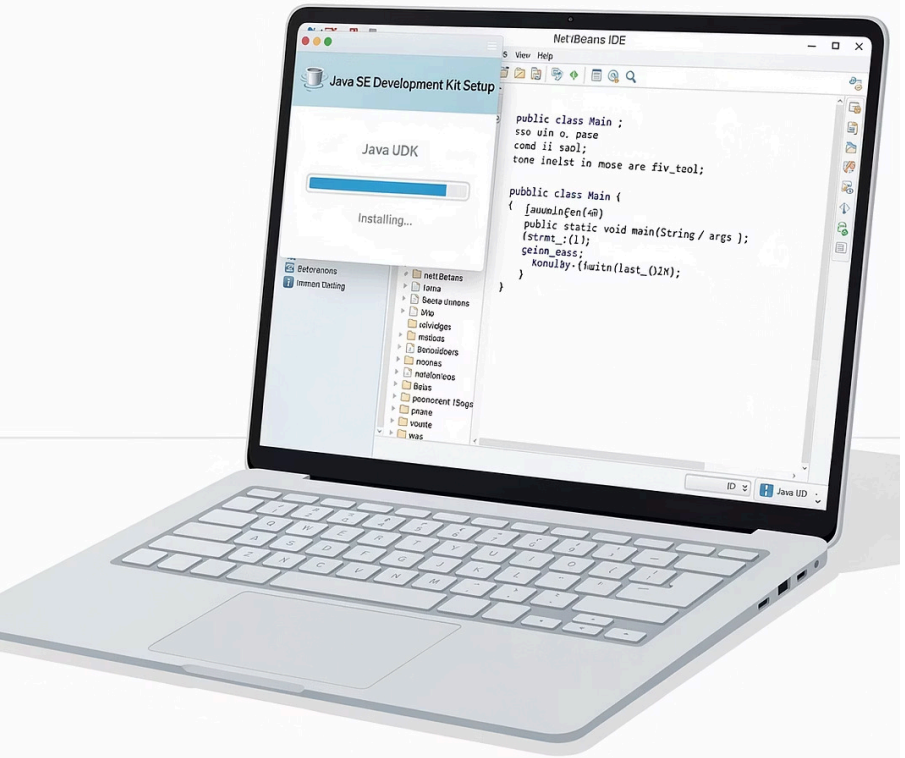
Bloque 4: Operadores y Lógica

Prioridad matemática, expresiones e instrucciones.

05

Bloque 5: Depuración Profesional

Breakpoints, Watch de variables y Prueba de Escritorio.



BLOQUE 1 — ENTORNO Y GESTIÓN

Configuración del Entorno: Java LTS y NetBeans

Java LTS (Long-Term Support)

Siempre usar la versión LTS más reciente (Java 21 o superior). Las versiones LTS garantizan soporte extendido y estabilidad para proyectos académicos y profesionales.

NetBeans IDE

NetBeans es el entorno oficial de la cátedra. Pasos de instalación:

1. Descargar el JDK LTS desde oracle.com
2. Instalar NetBeans desde netbeans.apache.org
3. Verificar que NetBeans detecte el JDK correctamente
4. Crear un proyecto nuevo: `File → New Project → Java Application`

BLOQUE 1 — ENTORNO Y GESTIÓN

Exportación e Importación de Proyectos (.zip)

Abrir el Proyecto

Asegúrate de que el proyecto compile sin errores antes de exportar.

Exportar como .zip

En NetBeans: `File → Export Project → To ZIP`. El archivo incluye todo el código fuente.

Entregar la Tarea

Subir únicamente el archivo `.zip` generado por NetBeans al aula virtual.

Importar un Proyecto

`File → Import Project → From ZIP`. Nunca descomprimir manualmente antes de importar.

⚠️ **Importante:** Solo se aceptan entregas en formato `.zip` .

BLOQUE 2 — COMENTARIOS Y SINTAXIS

Comentarios: Documentación y Herramienta de Debugging

Tipos de Comentarios en Java

Comentario de línea

`// Este es un comentario de una línea`

Comentario de bloque

`/* Este comentario abarca múltiples líneas */`

Comentario Javadoc

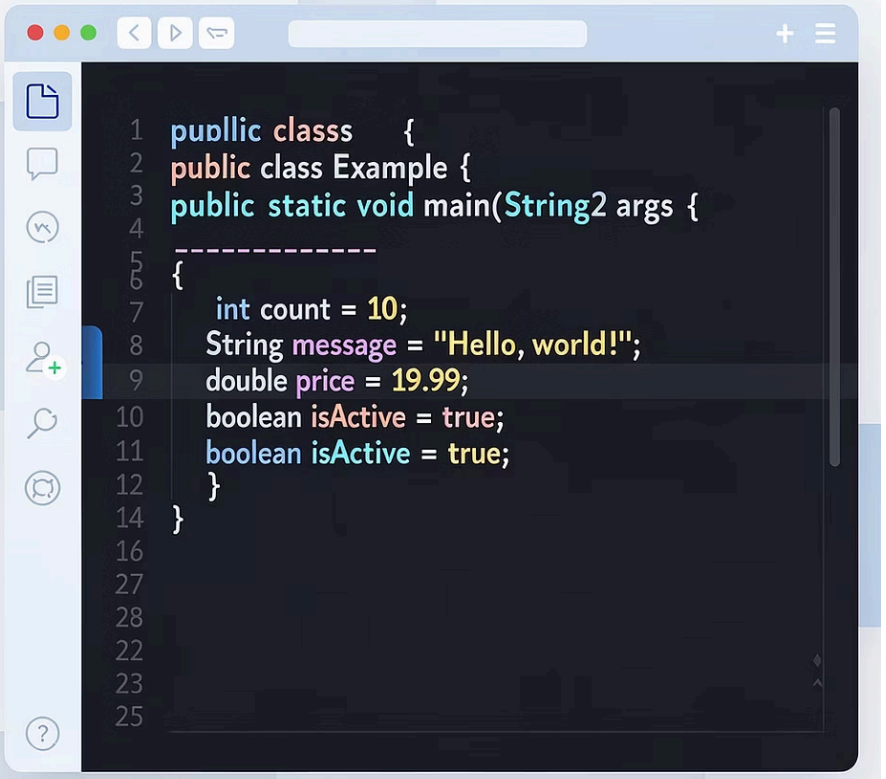
`/** Documenta clases y métodos públicos */`

Comentarios como "Borrador" en Debugging

Una técnica fundamental es **comentar temporalmente** líneas de código para aislar errores. Si una sección falla, comenta el bloque sospechoso y prueba el resto.

```
// System.out.println(resultado); // ← Comentado para probar
System.out.println("Valor intermedio: " + x);
```

Esto permite identificar exactamente en qué línea se produce el error sin borrar código.



BLOQUE 2 — COMENTARIOS Y SINTAXIS

Variables: Declaración vs. Inicialización

Declaración

Se reserva espacio en memoria para la variable, pero **no se le asigna un valor**. La variable existe pero no puede usarse aún.

```
int edad;
double sueldo;
String nombre;
```

Inicialización

Se le asigna un valor por primera vez a la variable declarada. A partir de este momento, la variable puede usarse en expresiones.

```
edad = 25;
sueldo = 1500.50;
nombre = "Ana";
```

Declaración + Inicialización

Es posible combinar ambas operaciones en una sola línea. Esta es la forma más común y recomendada cuando el valor inicial es conocido.

```
int edad = 25;
double sueldo = 1500.50;
String nombre = "Ana";
```

BLOQUE 3 — ESTÁNDAR DEL SCANNER

Configuración del Scanner

Paso 1: Importar la clase

Antes de usar Scanner, se debe importar la clase al inicio del archivo, antes de la declaración de la clase principal.

```
import java.util.Scanner;
```

Paso 2: Crear la instancia

Se crea un objeto Scanner vinculado a `System.in` (la entrada estándar del teclado).

```
Scanner sc = new Scanner(System.in);
```

Buenas Prácticas

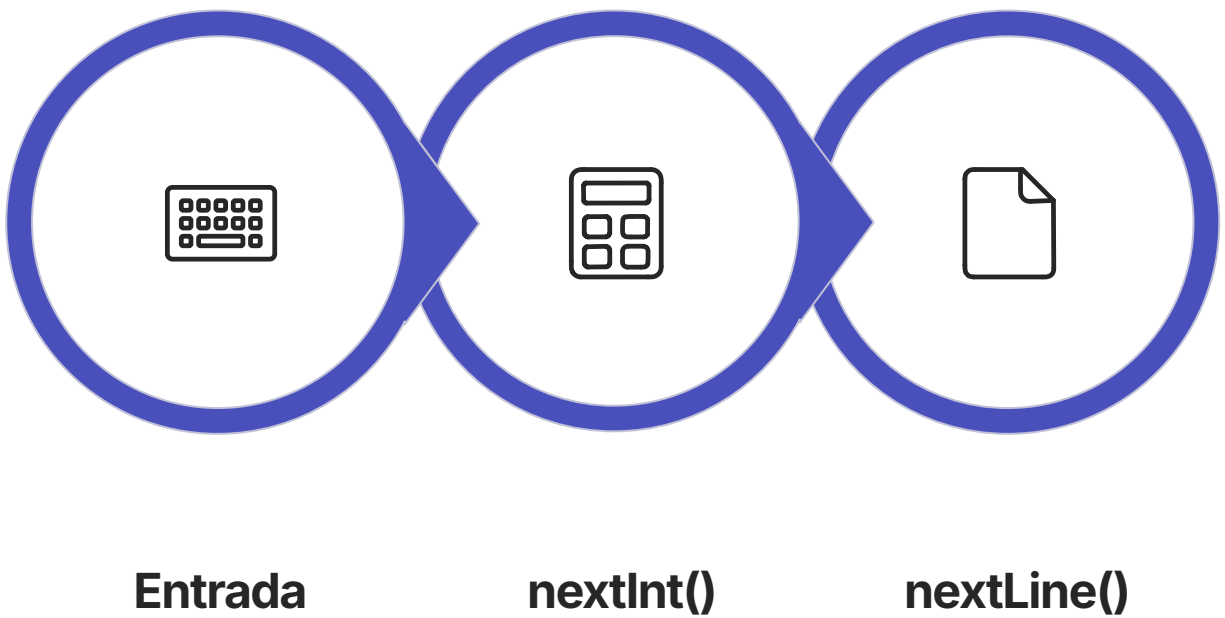
- Nombrar el objeto `sc` por convención de la cátedra.
- Declarar el Scanner al inicio del método `main`.
- Usar siempre `System.in` como fuente de entrada.
- Cerrar el Scanner al final del programa con `sc.close()`.

```
import java.util.Scanner;

public class MiPrograma {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);
 // ... código ...
 sc.close();
 }
}
```

BLOQUE 3 — ESTÁNDAR DEL SCANNER

El Problema del "Salto de Línea Fantasma"

⚠️ **Advertencia de Cátedra:** Este es el error más común al usar `nextInt()` o `nextDouble()` directamente. Comprender este problema es obligatorio.



Cuando se usa `nextInt()`, este método lee el número pero **deja el carácter de nueva línea (`\n`) en el búfer**. El siguiente `nextLine()` consume ese `\n` fantasma en lugar de esperar al usuario, produciendo una cadena vacía inesperada.

BLOQUE 3 — ESTÁNDAR DEL SCANNER

La Solución Definitiva: Estándar de Cátedra

🚨 **Requisito Obligatorio de Cátedra: NUNCA** usar `nextInt()` ni `nextDouble()` directamente. Siempre capturar con `sc.nextLine()` y realizar el parseo manual.

❌ Método Incorrecto

Usar métodos específicos de tipo genera el problema del salto de línea fantasma y es **rechazado por la cátedra**.

```
int edad = sc.nextInt();
double sueldo = sc.nextDouble();
String nombre = sc.nextLine(); // ← Lee vacío!
```

✅ Método Correcto (Estándar)

Leer siempre con `nextLine()` garantiza que el búfer se limpie completamente en cada lectura.

```
String edadStr = sc.nextLine();
String sueldoStr = sc.nextLine();
String nombre = sc.nextLine(); // ← Funciona!
```

BLOQUE 3 — ESTÁNDAR DEL SCANNER

Conversión de Datos: Parsing

Después de capturar la entrada como `String` con `nextLine()`, se convierte al tipo de dato necesario usando los métodos de parseo de Java.

Tipo Destino	Método de Parseo	Ejemplo
`int`	`Integer.parseInt()`	`int edad = Integer.parseInt(sc.nextLine());`
`double`	`Double.parseDouble()`	`double sueldo = Double.parseDouble(sc.nextLine());`
`long`	`Long.parseLong()`	`long cedula = Long.parseLong(sc.nextLine());`
`float`	`Float.parseFloat()`	`float temp = Float.parseFloat(sc.nextLine());`
`boolean`	`Boolean.parseBoolean()`	`boolean activo = Boolean.parseBoolean(sc.nextLine());`

💡 **Tip:** Si el usuario ingresa texto donde se espera un número, `parseInt()` lanzará una excepción `NumberFormatException`. Esto es útil para detectar entradas inválidas.

BLOQUE 3 — ESTÁNDAR DEL SCANNER

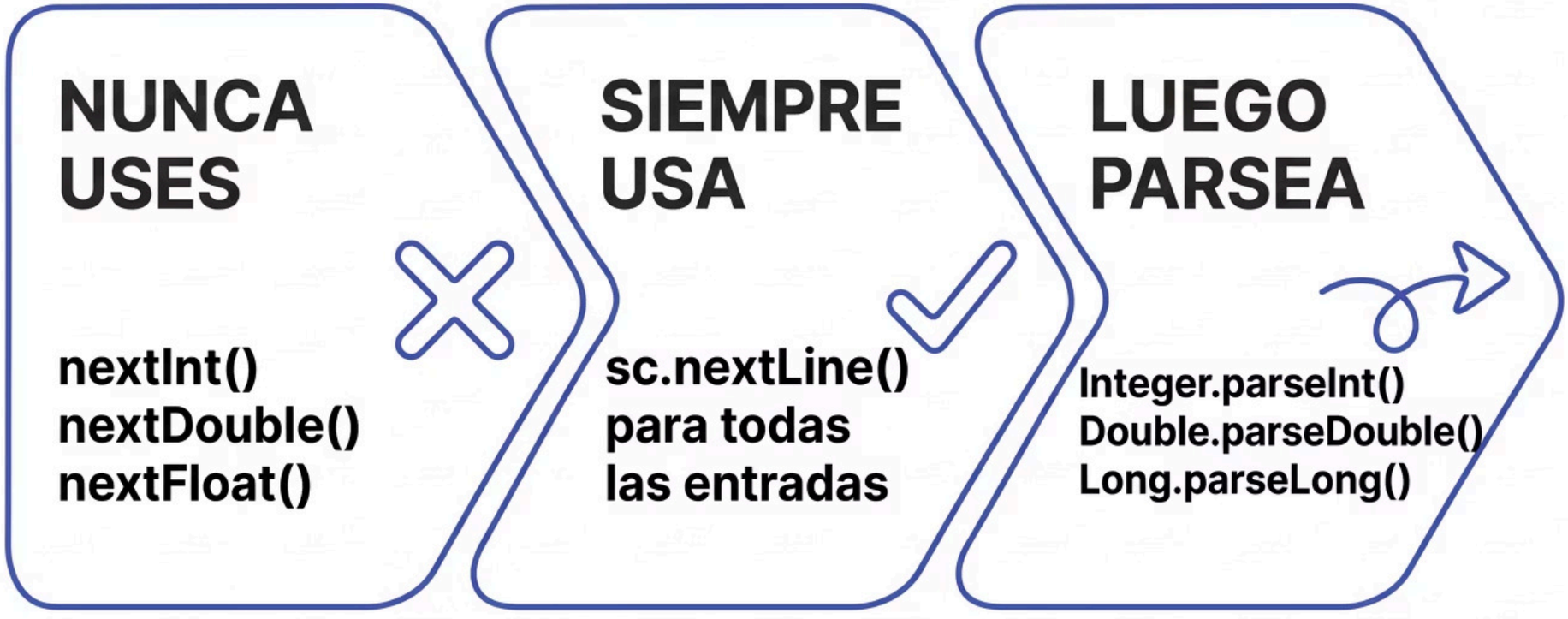
Ejemplo de Código Robusto con nextLine()

✅ **Estándar de Cátedra aplicado:** Este programa usa exclusivamente `sc.nextLine()` para todas las entradas y parseo manual para los tipos numéricos.

```
import java.util.Scanner;

public class DatosPersona {
 public static void main(String[] args) {
 Scanner sc = new Scanner(System.in);

 // Lectura de Nombre (String) - no requiere parseo
 System.out.print("Ingrese su nombre: ");
 String nombre = sc.nextLine();

 // Lectura de Edad (int) - parseo con Integer.parseInt()
 System.out.print("Ingrese su edad: ");
 int edad = Integer.parseInt(sc.nextLine());

 // Lectura de Sueldo (double) - parseo con Double.parseDouble()
 System.out.print("Ingrese su sueldo: ");
 double sueldo = Double.parseDouble(sc.nextLine());

 System.out.println("Nombre: " + nombre);
 System.out.println("Edad: " + edad + " años");
 System.out.println("Sueldo: $" + sueldo);

 sc.close();
 }
}
```

Resumen Visual: El Estándar del Scanner

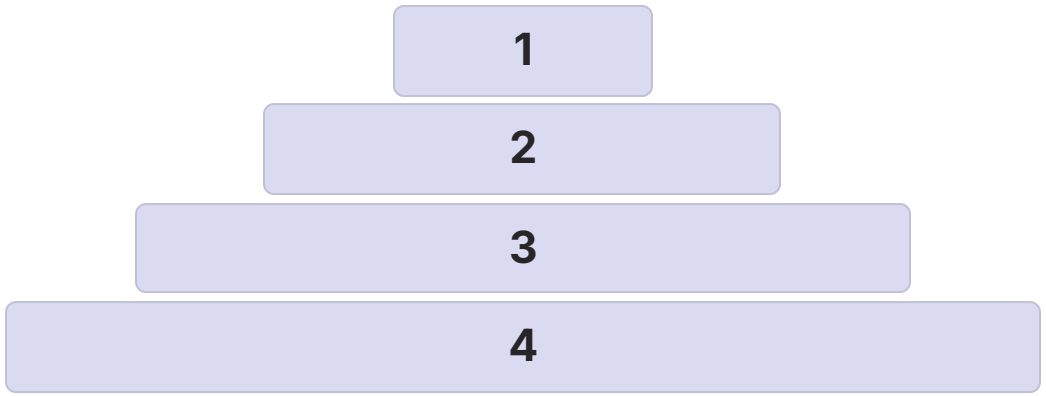
🚨 **Recordatorio de Cátedra:** El incumplimiento del estándar `nextLine()` + parseo manual es causal de descuento en la calificación.



El flujo correcto es siempre: **Capturar con nextLine()** → **Parsear al tipo necesario**. Este estándar elimina completamente el problema del salto de línea fantasma y produce código más robusto y predecible.

Prioridad Matemática y Uso de Paréntesis

Orden de Precedencia en Java



1. 1° Paréntesis `()`
2. 2° Potencia / Unarios
3. 3° `*` `/` `%`
4. 4° `+` `-`

Errores Comunes en Cálculos

Promedio incorrecto:

```
double prom = a + b + c / 3; //
```

BLOQUE 4 — OPERADORES Y LÓGICA

Expresión vs. Instrucción: Error "Not a Statement"

Expresión

Una expresión **produce un valor** pero no realiza una acción por sí sola. No puede estar sola en una línea de código.

```
a + b       // Expresión: calcula pero no hace nada
x * 2 + 1   // Expresión: solo un valor
```

Instrucción (Statement)

Una instrucción **realiza una acción**. Termina siempre con punto y coma ;. Puede contener expresiones.

```
int suma = a + b; //
```

BLOQUE 5 — DEPURACIÓN PROFESIONAL

Depuración con Breakpoints y Watch en NetBeans

Breakpoints (Puntos de Interrupción)

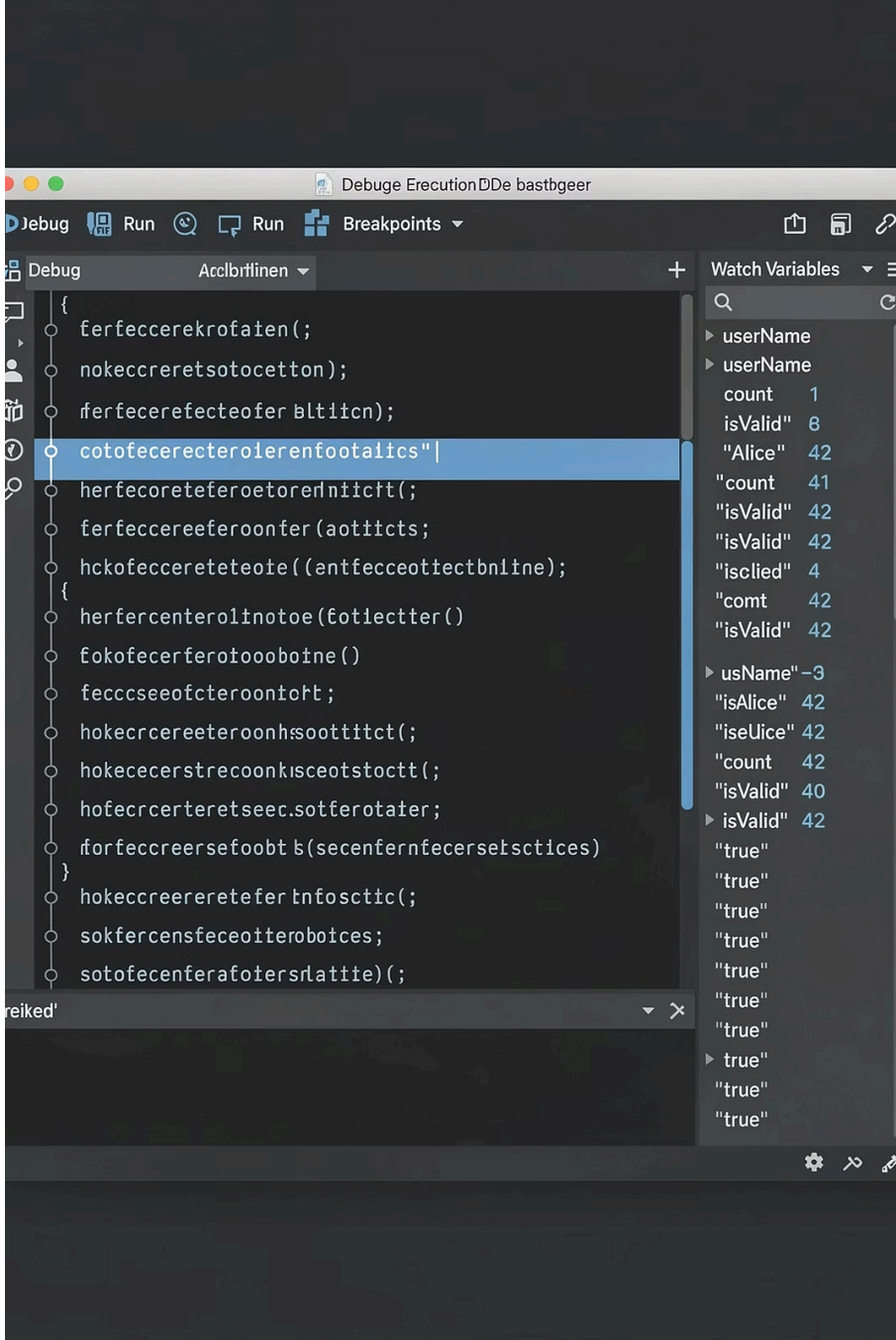
Haz clic en el margen izquierdo de una línea para colocar un breakpoint (aparece un círculo rojo). Al ejecutar en modo Debug (`Shift+F5`), el programa se detiene en esa línea, permitiendo inspeccionar el estado.

Watch de Variables

En el panel **Variables/Watches** del debugger, puedes agregar variables para monitorear su valor en tiempo real mientras el programa avanza línea por línea con `F8` (Step Over).

Controles del Debugger

`F8` avanza línea por línea. `F7` entra dentro de un método. `F5` continúa hasta el siguiente breakpoint. Estos controles permiten seguir el flujo exacto del programa.



BLOQUE 5 — DEPURACIÓN PROFESIONAL

La Técnica de "Prueba de Escritorio"

¿Qué es la Prueba de Escritorio?

Es la simulación **manual y en papel** de la ejecución de un programa. El programador actúa como la computadora, siguiendo cada instrucción y registrando el valor de cada variable en una tabla.

Es la técnica más fundamental de depuración y no requiere ninguna herramienta tecnológica.

Cómo Realizarla

1. Escribe los nombres de todas las variables como columnas de una tabla.
2. Lee el código línea por línea de arriba hacia abajo.
3. En cada asignación, actualiza el valor en la tabla.
4. En cada condicional, evalúa la condición con los valores actuales.
5. Compara el resultado final con el resultado esperado.

Si los valores en la tabla no coinciden con lo esperado, encontraste el error.

BLOQUE 5 — DEPURACIÓN PROFESIONAL

Ejemplo de Prueba de Escritorio

```
int a = 5;
int b = 3;
int resultado = a * b + 2;
System.out.println(resultado);
```

Línea	Instrucción	a	b	resultado
1	`a = 5`	5	—	—
2	`b = 3`	5	3	—
3	`resultado = 5*3+2`	5	3	17
4	`println(17)`	5	3	17

La tabla muestra que `resultado` es **17**. Si esperábamos otro valor, la línea 3 es donde debemos revisar la prioridad de operadores.

BLOQUE 5 — DEPURACIÓN PROFESIONAL

Comparativo de Técnicas de Depuración

Comentar Código

Aislar secciones comentándolas para identificar el bloque problemático. Rápido y sin herramientas.

println() de Diagnóstico

Imprimir valores intermedios con `System.out.println()` para rastrear el flujo. Eliminar antes de entregar.

Prueba de Escritorio

Simulación manual en papel. Ideal para entender la lógica antes de codificar o para errores de lógica complejos.

Debugger de NetBeans

Breakpoints + Watch. La herramienta más poderosa para inspeccionar el estado exacto del programa en ejecución.

✅ Checklist: Antes de Entregar tu Tarea

- [ ] **¿Usé `nextLine()` para todas las entradas?**

  Verifica que no exista ningún `nextInt()`, `nextDouble()` ni `nextFloat()` directo en tu código. Todo debe pasar por `sc.nextLine()` con parseo manual.

- [ ] **¿El programa compila y ejecuta sin errores de lógica?**

  Realiza una prueba de escritorio o usa el debugger para verificar que los resultados sean los esperados. Un programa que compila pero da resultados incorrectos **tiene errores de lógica**.